

AADK2 – TASK 3

Student Name: Rajat Prakash Dhal

Course/Unit: Android Development with Kotlin

Android Studio Installed: 16 February 2026

Project Created & Run: 22 February 2026

Project Name: testing3

In Jetpack Compose (used in **Android Studio**), a **Modifier** is used to change how a composable looks, behaves, or is positioned. Modifiers control size, spacing, alignment, background, and responsiveness.

In this report, I experimented with three modifiers to understand how they affect layout across different screen sizes.

Experiment 1 — padding()

- **Mental Expectation**

Padding should create space **inside** the composable boundary, pushing content inward.

Test Setup

- Simple Text inside a Box
- Compared: No padding vs padding(16.dp)
- Tested on:
 - Compose Preview
 - Small phone emulator
 - Larger screen (tablet/large phone)

What I Expected vs What Happened

Expected	What Happened
Content shifts inward	Content moved inward correctly
Outer size remains same Size remained same unless combined with fill modifiers	

- **Observation**

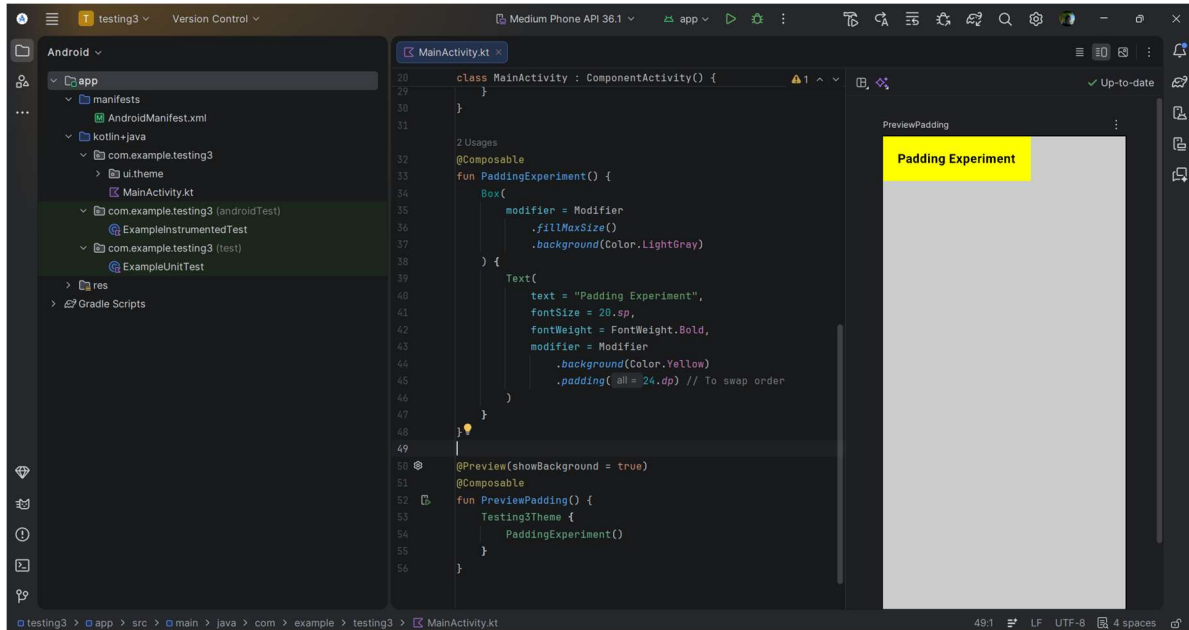
On larger screens, padding visually appears smaller because the screen is wider, but spacing remains constant (16dp).

- **Modifier Order Note**

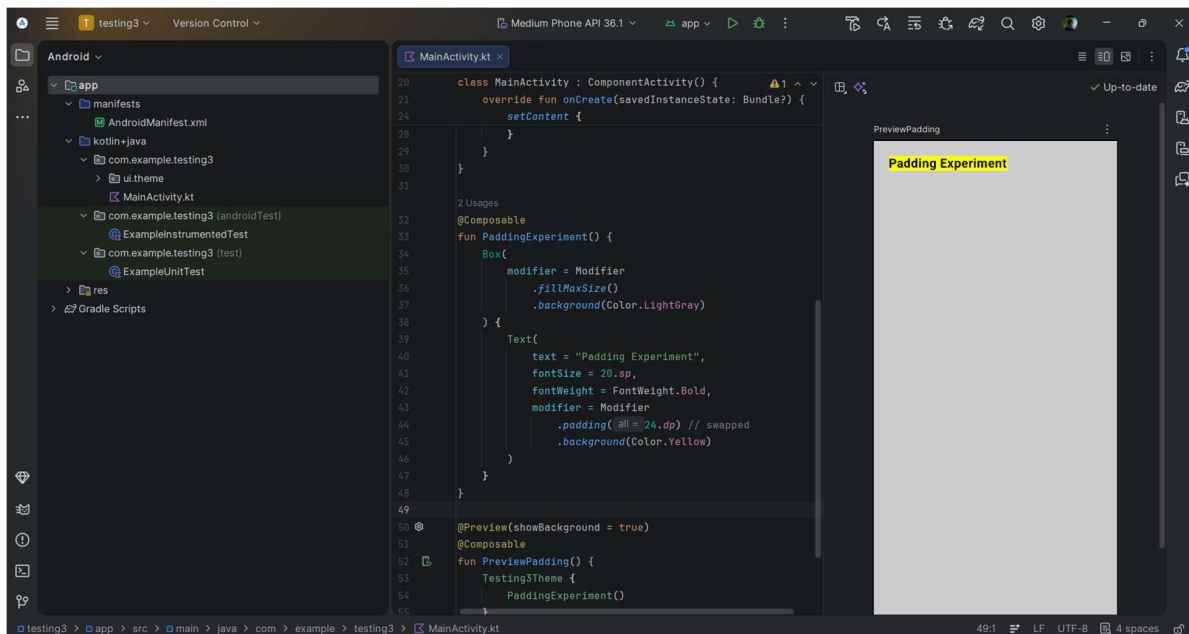
background().padding() → Background does NOT include padding area.

padding().background() → Background includes padded area.

.background > .padding



.padding > .background



Experiment 2 — fillMaxWidth() + weight()

- **Mental Expectation**

fillMaxWidth() should stretch the composable across the screen .weight() should distribute available space proportionally.

Test Setup

- Used a Row with two buttons
- Compared:
 - Without weight
 - With weight(1f) on both buttons

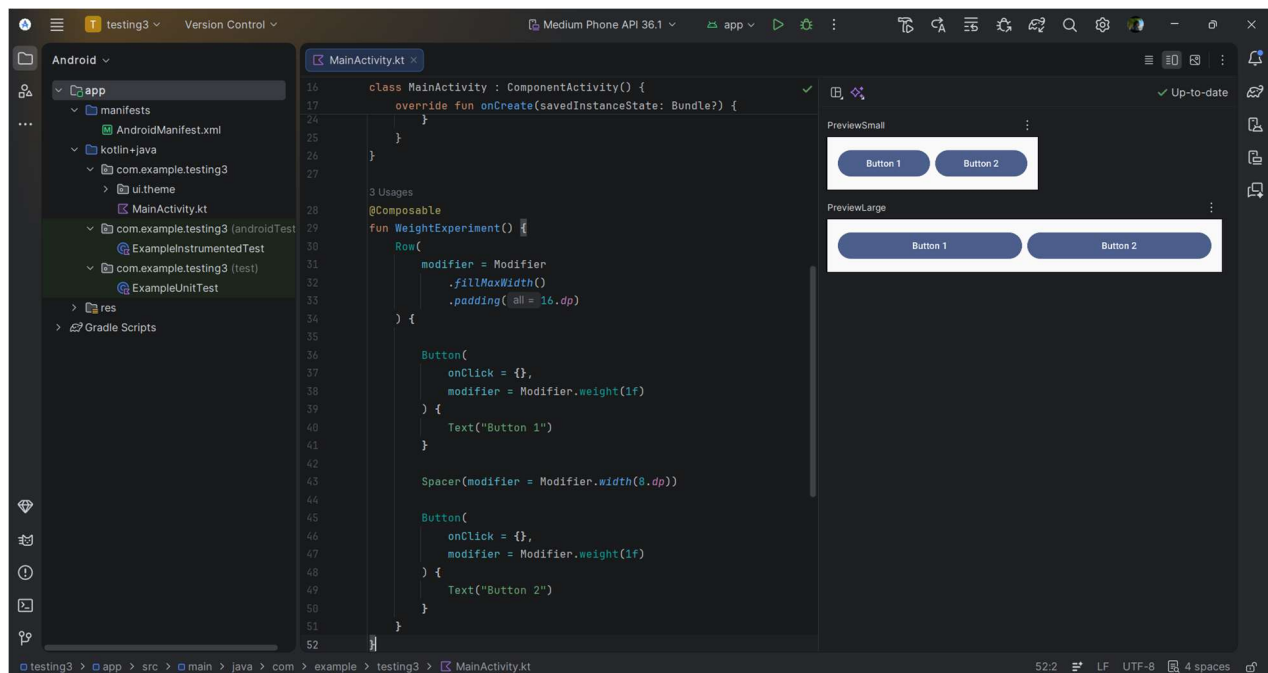
What I Expected vs What Happened

Expected	What Happened
Buttons share equal width	Buttons resized equally
Larger screen gives more space	Buttons expanded proportionally

Observation

On tablet/large screens, weighted elements expand significantly. Without weight, buttons stay compact.

.fillmaxwidth() + weight()



Experiment 3 — align() & Arrangement.SpaceBetween

- **Mental Expectation**

align() should control child position inside parent.

Arrangement.SpaceBetween should push items to opposite ends.

Test Setup

- Used a Column with 3 text elements
- Applied Arrangement.SpaceBetween
- Tested alignment variations

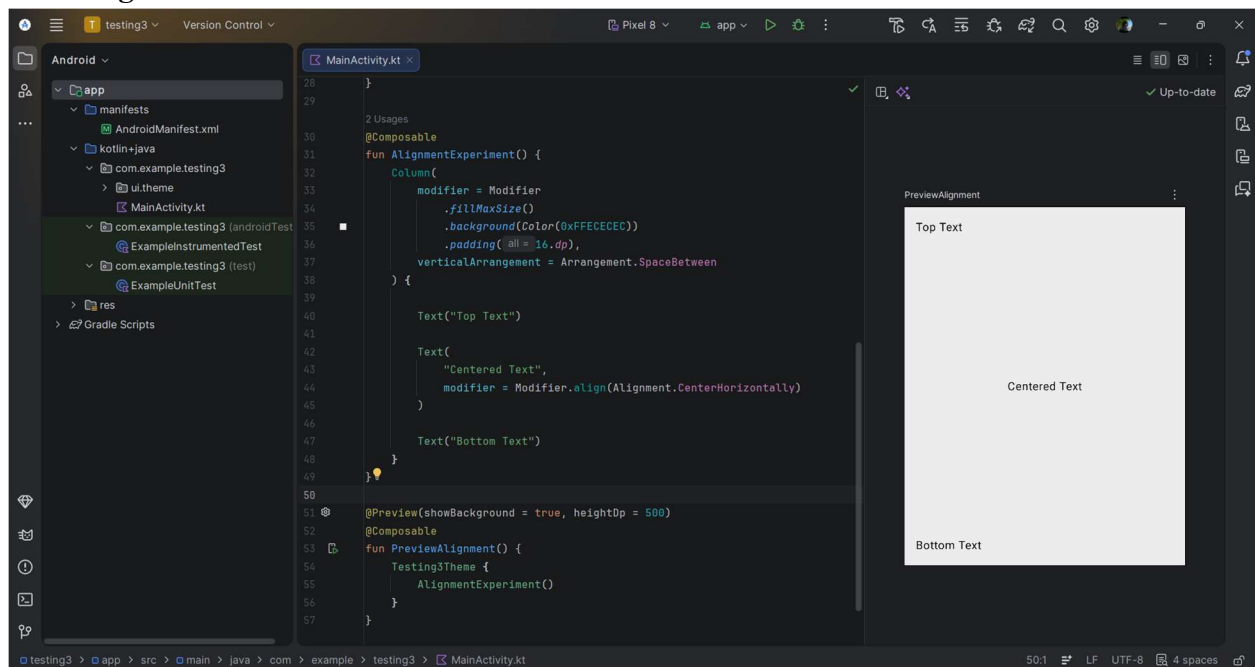
What I Expected vs What Happened

Expected	What Happened
Items move apart vertically	Items spread evenly
Alignment changes position	Items shifted correctly

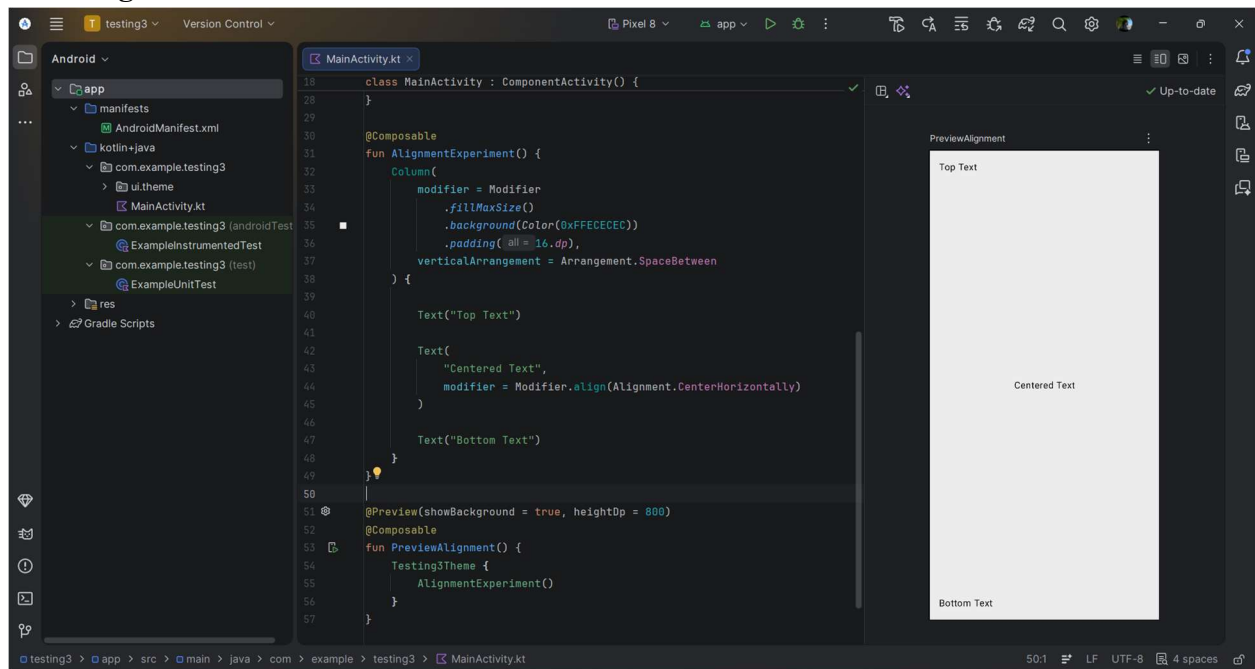
Observation

On larger screens, spacing increases dramatically when using SpaceBetween, making layouts look too stretched.

with heightDP=500



with heightDP=800



Responsive Behavior Comparison

Modifier	Small Screen	Large Screen	Key Insight
padding	Fixed spacing	Same dp, visually smaller	dp is density-based
fillMaxWidth	Fills width	Fills more width	Expands with screen
weight	Shares space	Expands proportionally	Great for flexible UI
SpaceBetween	Moderate gap	Large gap	Can over-stretch UI

Responsive Design Suggestions (Compose)

Based on my experiments with `padding()`, `weight()`, `fillMaxWidth()`, and alignment modifiers, here are three practical tips for designing responsive layouts in Jetpack Compose:

1. Use `weight()` for Proportional Widths

When placing components inside a `Row` or `Column`, using `Modifier.weight()` ensures elements expand proportionally to available space.

Why important?

- Prevents uneven spacing
- Adapts automatically to larger screens
- Makes layouts flexible across phone and tablet sizes

👉 Example: Two buttons with `weight(1f)` share equal width on both small and large devices.

2. Test on Multiple Screen Sizes

Always test layouts using:

- Compose Preview with different `widthDp/heightDp`
- Small phone emulator
- Larger phone or tablet emulator

Why important?

Layouts may look balanced on small screens but overly stretched on larger ones (especially with `Arrangement.SpaceBetween`).

3. Use Arrangement and Alignment for Controlled Spacing

Instead of manually adding many `Spacer()` elements, prefer:

- `Arrangement.SpaceBetween`
- `Arrangement.SpaceEvenly`
- `Alignment.CenterHorizontally`

Why important?

These provide consistent and predictable spacing across devices.

Accessibility Check (Mini Review)

Experiment Reviewed: `fillMaxWidth()` + `weight()` (Two Buttons Layout)

Potential Accessibility Issue:

If buttons become too small on compact screens or too close together, users with larger fingers may struggle to tap accurately.

Suggested Fix:

- Ensure minimum height using:

```
modifier = Modifier
```

```
.weight(1f)
```

```
.height(48.dp)
```

- Maintain adequate spacing using:

```
Spacer(modifier = Modifier.width(12.dp))
```

- Follow recommended **minimum touch target size of 48dp**.